MTU
Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

STAT8010: Intro to R for Data Science

Chapter 1.

Introduction to Base R

Francisco Hernández

francisco.hernandez@cit.ie

Why use R?

MTU
Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

R was developed from another statistical **language S**, by Ihaka and Gentlemen in the early **90s**.
R is **free** and open-source so many people are contributing to its development.
R is **superior** in many ways to existing commercial products such as **SAS, SPSS or Stata**.
It is **available** for Windows, Mac and Linux
Employers like R - one of the most requested software by graduate employers
Hit the ground running in a **data science** role
Exploratory Analysis
Performance Analysis
Another important **aim** of this course is to help you become **self-sufficient** at R-based development. This includes learning to be organised and find helpful resources, but also building up self-confidence by exploring a variety of examples.

Download R

The Comprehensive R Archive Network (r-project.org)

CRAN
Mirrors
What's new?
Task Views
Search

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Other

Documentation
Manuals
FAQs
Contributed

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- Download R for Linux (Debian, Fedora/Redhat, Ubuntu)
- Download R for macOS
- Download R for Windows

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

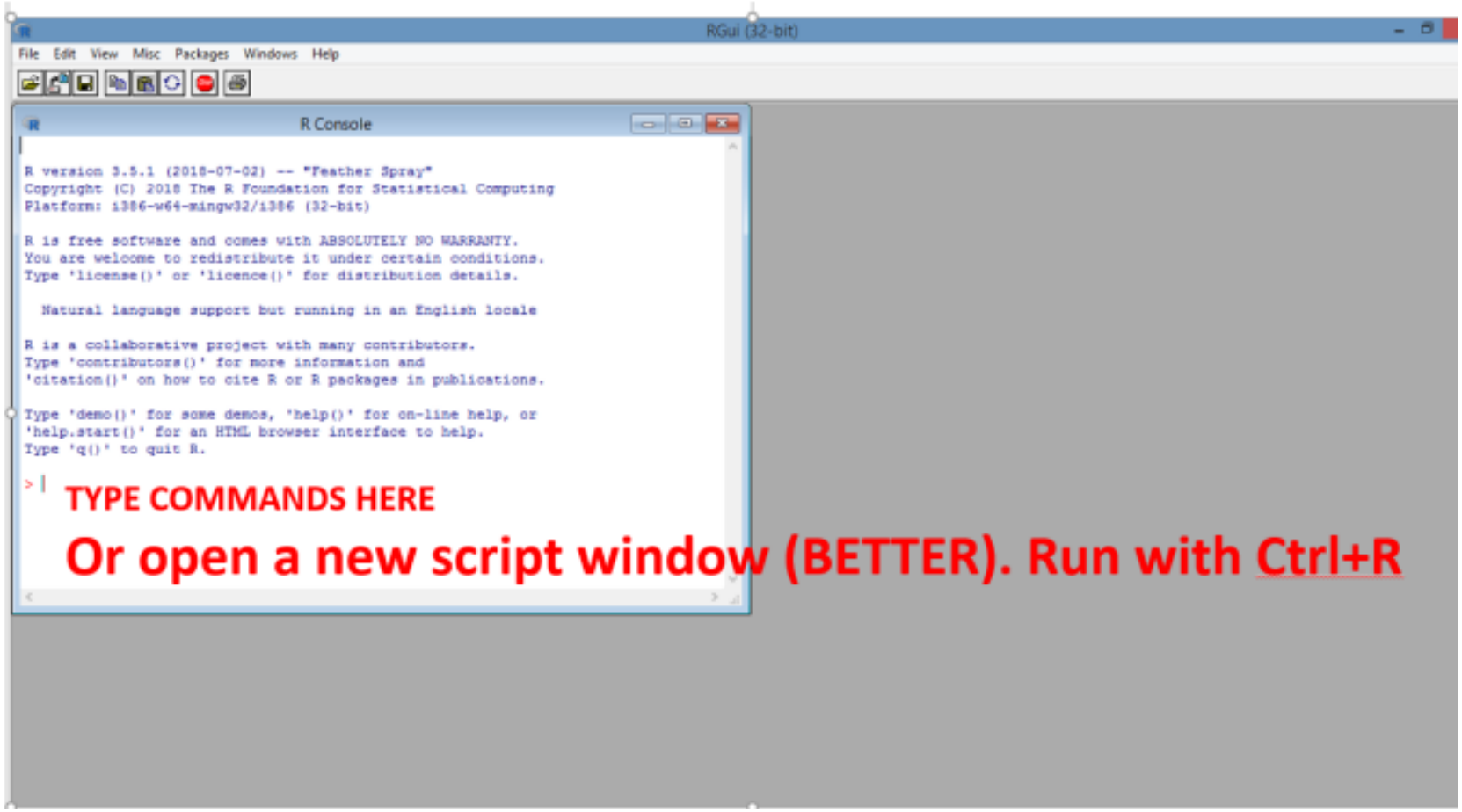
Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2021-08-10, Kick Things) R-4.1.1.tar.gz, read what's new in the latest version.
- Sources of R alpha and beta releases (daily snapshots, created only in time periods before a planned release).
- Daily snapshots of current patched and development versions are available here. Please read about new features and bug fixes before filing corresponding feature requests or bug reports.
- Source code of older versions of R is available here.
- Contributed extension packages

Questions About R

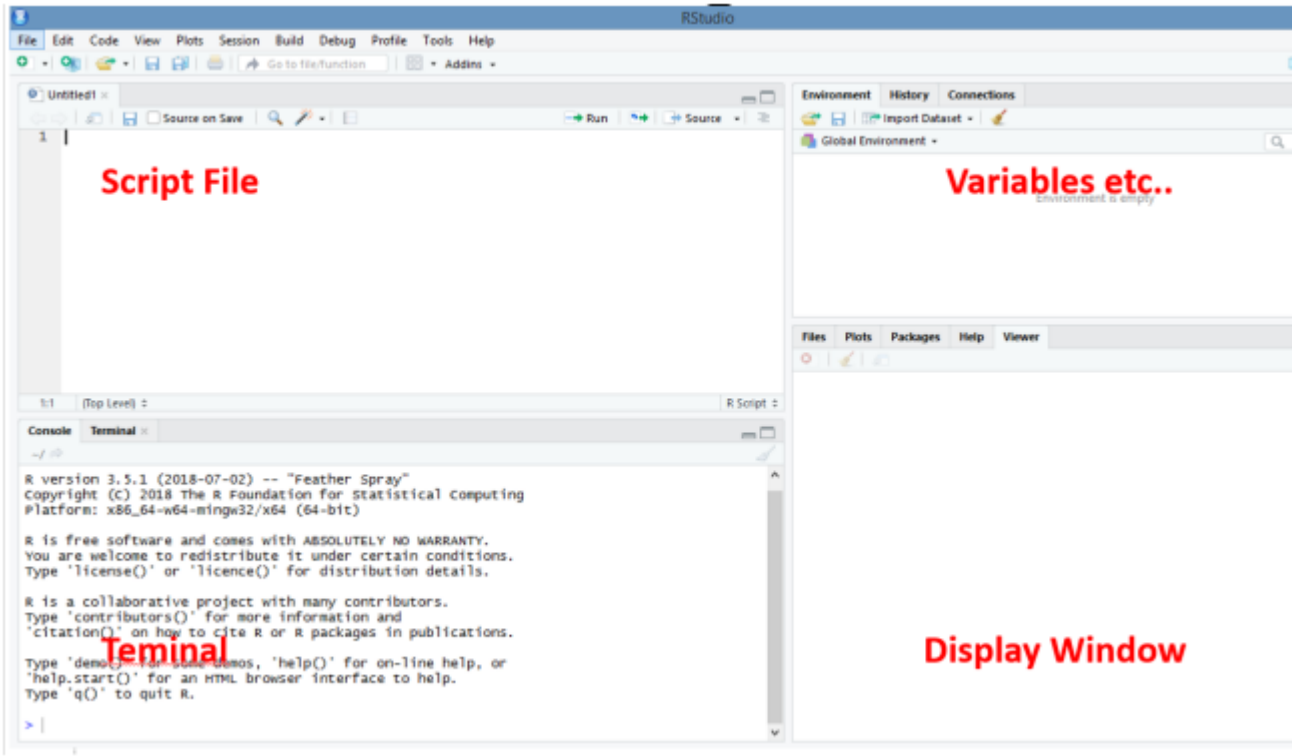
- If you have questions about R like how to download and install the software, or what the license terms are, please read our answers to frequently asked questions before you send an email.

Launch R


RGui (32-bit)
File Edit View Misc Packages Windows Help
R Console
R version 3.5.1 (2018-07-02) -- "Feather Spray"
Copyright (C) 2018 The R Foundation for Statistical Computing
Platform: i386-w64-mingw32/i386 (32-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
Natural language support but running in an English locale
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
> TYPE COMMANDS HERE
Or open a new script window (BETTER). Run with Ctrl+R

Download RStudio

Download the RStudio IDE - RStudio



Succeeding Together

www.mtu.ie

Assignment in R

R allows for two different versions of assigning a value to a variable. **“ = “** and **“ <- “**
You will obtain the same answer (most of the time)
“ <- “ was the **original** assignment version
“ = “ was introduced more recently.

There is a subtle difference between the two (you can easily use either version 99% of the time).

The **differences** arise with code like the following:

a = b <- 4
c <- d = 4

```
> a = b <- 4
> c <- d = 4
Error in c <- d = 4 : could not find function "<-<-"
> a;b
[1] 4
[1] 4
```

Help functions in R

MTU
Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

R has **lots of functions** which do many different things to data.

If you **know the name** of the function you are interested in, you can type `help(sum)`

This will provide you with the **help** file for that function, which will outline: 1. what the function does; 2. its syntax ; and 3. built in options.

If you **do not know** the function name you can type: `help.search('standard deviation')`

This will provide you with a **list of functions** which fall under this topic. You can also do a general search from the display window (bottom right command).

You can use **shortcuts** for `help (?)` and `help.search (??)`

Another useful functions are:

example: gives a list of examples for a specified function. `example(mean)`

demo: gives an example of the range of things R can do. `demo(graphics)`

Object Types

R is an **object** orientated language. When R does anything it creates and manipulates objects. The basic ones are:

Vectors: These are one-dimensional sequences of elements of the same mode. (More on modes later.) e.g. a vector of length 26 where each element is a letter of the alphabet.

Arrays & Matrices: These are 2-dimensional rectangular objects (matrices) and higher dimensional rectangular objects (arrays). All elements of matrices and arrays have to be of the **same mode**.

Factors: vectors to classify categorical data. They behave differently than vectors containing **numerical, integer or character** elements.

Object Types

Lists: like vectors but they do **not** have to contain elements of the **same mode**. The first element of a list could be a vector of the 26 letters of the alphabet. The second could contain a vector of the prime numbers below 1000. The third could be a 2 by 7 matrix.

Data Frames: best understood as *special matrices* (they are technically a type of list). For most applications involving datasets, data frames will be used. The are **two dimensional** containers with rows corresponding to *'observations'* and columns corresponding to *'variables'*.

Functions: objects that take other objects as inputs and return some new object..

Mode/Class Types

All R objects also have a **mode** or **class**. The mode describes the *type of data* or information contained in the object.

Integer: Integer values (1, 2, -4, -598, etc.)
Numeric: Real numbers (2, 97, 487.1, -567.4, etc.)
Complex: Complex or Imaginary numbers (1 + 2i, –3 – 4i, 3.4 – 2.7i, etc.)
Character: Elements made of text strings (”text”, ”Hello”, ”1234”, etc.)
Logical: Data containing logical constants (TRUE and FALSE)
Raw: (x =charToRaw(“Hello”))

You can check the type of data by: `is(x)` or `print(class(x))`

Coercing Mode Types

Oftentimes, **errors** are caused by variables that are in the **wrong mode** or class.
For example, if a function expects an integer but you feed in a numeric, an error can occur.
This error might not be immediately obvious, as the act of changing the object between mode types can change how the function behaves with the output.

One way to check the object type you are dealing with it is by using `is()`.

```
x <- c(1,2,3)
is(x)
```

Coercing Mode Types

It is also possible to **force objects** into certain modes using the below commands:
as.numeric(), as.integer(), as.character(), as.logical(), as.complex(), many more...

```
a <- 1.345
b <- 2.678
c <- 0
as.integer(a); as.integer(b); as.integer(c)
as.character(a); as.character(b); as.character(c)
as.logical(a); as.logical(b); as.logical(c);
```

Try playing around with the different coercion commands to get a feel for how they change objects.

Vectors & Data

Vectors & matrices are the **simplest** way of storing data in R.
R handles these objects differently than other programming languages.
More complex data objects are stored in **lists** and **dataframes**.
Vectors can be numeric or text, but must always be of the **same mode** or type, i.e. you can't mix text with numbers.
Use the mode function to tell you the type of values stored in a vector.

```
t = c(1,2,4)
pets = c('cat','dog','rabbit')
mode(t)
mode(pets)
```

Vectors Operations

The "`c()`" command is used to create a vector.

For example `x = c(0.1, 2, 4.3, 3.1, 5)`

x[3] returns the third value in that vector, i.e. 4.3

x[1:3] returns the first to third values inclusive

x[c(1,4)] returns the first and fourth values.

x[-(1:3)] returns all but the first to third values

x[x>3] selects the subset of values in x that are greater than 3

N = length(x) stores the length of x in the variable N

You can also use **indexing to reassign** certain values within a vector: x[2] <- 22

Similarly, you can **extend a vector** with new values: x[6] <- 7

What does x now return? Notice the change we've made

`replace()` function

```
replace(x, c(3, 6), 99)
```

Adding & Deleting Values in a Vector

If you want to some value to the **middle of a vector**, without overriding the current values, you can use indexing as shown below:

```
x = c(88, 5, 12, 13, 20)
x = c(x[1:3], 168, x[4:5])
```

This inserts 168 in the 4th index and creates a new vector of length 6.

To return to the original x, we can **delete** the fourth element:

```
x = x[-4]
```

There are similar commands to add columns and rows to a matrix, using the functions ***cbind*** and ***rbind*** respectively.

Vectors Operations

Standard **arithmetic** applies to vectors of **equal length**, e.g.

```
v <- 2*x+y+1
sum((x-mean(x))^2)/(length(x)-1)
```

Regular **sequences** can be generated easily:

```
s1 = seq(-5, 5, by=.2)
s2 = seq(from=-5, length=51, by=.2)
xx = rep(x, times = 2)
xxx = rep(x, each = 3)
```

Vectors may also contain **characters** or strings:

```
fruit <- c(5, 10, 4)
names(fruit) <- c("orange", "banana", "apple")
lunch <- fruit[c("apple", "orange")]
```

Matrix Operations

First create a vector:

```
x=c(1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Now make it a 3x3 matrix:

```
M = matrix(x,ncol = 3)
```

Notice that this creates the matrix column-wise

If you prefer to create the matrix **by rows**:

```
M = matrix(x, ncol = 3, byrow = TRUE)
```

The dimensions of the matrix are given by:

```
dim(M)
ncol(M)
nrow(M)
```

Matrix Operations

Transpose matrix: `t(M) or aperm(M,c(2,1))`

Diagonal of matrix: `diag(M)`

Determinant of matrix: `det(M)`

You need to be very careful when **dealing with products of matrices**!

Standard multiplication (*) and division (\ ) work element-wise.

Element-by-element product: `A * B` (if A and B are square)

Matrix product: `A %*% B`

Indexing

You can extract parts of a matrix via **indexing**, as in vectors.

To get the (2,3) element, which is 6:

M[2,3]

To get the entire second row:

M[2,]

To get the entire third row:

M[,3]

The below command gives the second and third rows and the first column of M:

M[2:3,1]

Arrays

Make a vector into an array using

`array(vector,dim=...)`

For example, try the following code:

```
h = seq(1,24,by = 1)
Z <- array(h,dim = c(3,4,2))
Z
```

Now try indexing into Z using

```
Z[1,1,1]
Z[1,2,2]
```

```
> Z <- array(h,dim = c(3,4,2
> Z
, , 1

     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12

, , 2

     [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
[3,]   15   18   21   24
```

Lists

Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

Syntax: `lst <- list(name1 = object1,...,nameN = objectN lst[[index]]`

Example: Create a list

```
lst <- list(name="Fred", wife="Mary",no.children=3,child.ages=c(4,7,9))
```

Now access the list **components**; in this example:

`lst[[2]]` returns ”Mary”

`lst[[4]][1]` returns 4

Components may also be accessed by their name **using $**, e.g.

`lst$name` returns ”Fred”

`lst$child.ages[1]` returns 4

`length(lst)` gives the number of top level components

Data Frames

Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

Syntax: `D <- data.frame(name1 = arg1, ..., nameM = argM)`

A data frame is a list of class data.frame.
It can be seen as a **matrix** with columns of possibly **different types**/classes
Character vectors are coerced to **factors**
Vector components must all have **equal length**
Matrix structures must all have **equal row size**

NA & NULL values

R has two different placeholders for **missing/unknown** data, **NA** and **NULL**.
NA is used specifically for missing values. This can cause some issues with certain functions, unless you know how to deal with them:

```
X = c(88, NA, 12, 168, 13)
mean(x)
mean(x, na.rm=TRUE)

Y = c(88, NULL, 12, 168, 13)
mean(y)
```

```
> x= c(88, NA, 12, 168, 13)
> x
[1]  88  NA  12 168  13
> mean(x)
[1] NA
> mean(x, na.rm=TRUE)
[1] 70.25
>
> y= c(88, NULL, 12, 168, 13)
> y
[1]  88  12 168  13
> mean(y)
[1] 70.25
```

NA & NULL values

Consider the following simple code:

```
z=NULL
for(i in 1:5){z=c(z,i)}; z
```

Compare this with:

```
z=NA
for(i in 1:5){z=c(z,i)}; z
```

```
> z=NULL
> for(i in 1:5){z=c(z,i)}; z
[1] 1 2 3 4 5
> for(i in 1:5){z=c(z,i)}; z
 [1] 1 2 3 4 5 1 2 3 4 5
> z=NA
> for(i in 1:5){z=c(z,i)}; z
[1] NA  1  2  3  4  5
```

In the first example NULL literally doesn't exist. In the second the NA is treated as an unknown missing value and included in z.

Sourcing Data

Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

Sourcing Data It will often be the case that data are provided in one of the following formats:
an **excel** file,
a **text** file,

```
> getwd()
[1] "C:/Users/Francisco.Hernandez/OneDrive - Cork Institute of Tec
> list.files()
 [1] "Adobe"
 [2] "Cert Predictive Analytics - SpringBoard Validation 14 May  .
 [3] "Custom Office Templates"
 [4] "DATA8009_Assignement 1_2021.docx"
 [5] "Database1.accdb"
 [6] "desktop.ini"
 [7] "Minitab.mpx"
 [8] "Natural Science.docx"
 [9] "New folder"
[10] "OneNote Notebooks"
[11] "Python Scripts"
[12] "quine.txt"
[13] "R"
[14] "rpackages in 3.4.3.txt"
```

a comma separated value (**csv**) file,
another data file type.
R has a variety of **functions** available to import data in these formats.
If you set up a project in R: the default working directory is the project location.
This means that the project can ”see” any files in that folder and easily import them into R.
This is also the location where any created files are written.

Dealing with Directories

MTU
Ollscoil Teicneolaíochta na Mumhan
Munster Technological University

To check the working directory:

```
getwd()
```

You can change the working directory using:

```
setwd("file path")
```

You need to be careful of the format when specifying the file path.
For example, to introduce the file location `"C:\Desktop\STAT8010\"` into R you would need to **change the backslah (\)** for:

Double backslash ("\" → "\\")
One slash ("\" → "/")

Read in Data

`read.table()` reads in a file in table format and returns a data frame object

This function is particularly useful where data are in formats like **tab separated**.

The function is part of **base R** and is widely used `read.table("filename.txt")`

`read.csv()`, a variant of read.table() is particularly useful for reading CSV files

`read.xlsx()` & `read.xlsx2()` reads in a file in table format and returns a data frame object

This is not in base R but requires an **additional package** to be installed. The syntax is similar to read.table, i.e. `read.xlsx("filename.xlsx", Sheet="SheetName")`

library(foreign): `Opens files such as SPSS.`

Read in Data

`scan()` reads data from the console or from an input file into a vector or list

Scan can be particularly useful where data are in a format that is **not structured well**.

scan() is often called within `matrix()` when uploading data:

```
X <- matrix(scan("patient.dat"), ncol=4, byrow=TRUE)
```

```
Sometimes helpful:
skip
what
```

Write Data

`write.table()` creates a file and writes the data frame content in it

`write.csv()`, a variant of write.table() creates a CSV file

`write.xlsx()` creates an excel file with the data provided.

`Other options for SPSS, Stata, …`

Logical Operators

Among the most used features of R are the logical operators. They are crucial for all sorts of data manipulation.

When R evaluates statements containing **logical** operators it will return either TRUE/FALSE.

Below are the most used logical operators:

< less than	and	<= less than or equal to
> greater than	and	>= greater than or equal to
== equal	and	!= not equal

& and

| or (this is found using shift +\)

! not

Logical Expressions

Trying out these operators: Run the below code and note the outputs

```
1 == 1
1 == 2
1 != 2
1 <= 2 & 1 <= 3
1 == 1 | 1 == 2
1 > 1 | 1 > 2 & 3 == 3
1 > 1 & 1 > 2 & 1 > 3
```

Programming

R programming consists in **writing elaborate R instructions** in an R script, which can then be **re-used** at a later stage.

It is usually better to do it in an **R-dedicated editor** (e.g. RStudio).

It is strongly recommended to keep your code **clear and tidy**.

Indenting **blocks of instructions** is quite often time-saving and makes your code easier to read and follow.

Any code should also be annotated by **comments**, using #

Write code as if for someone else: anyone will struggle to understand his/her own code after some time, if this code is not well commented and organised

If statement

We mentioned the “if” statements before. We will now formalise this. Firstly before looking at code we will consider the logic of an if statement. ”if” is used as follows:

Syntax:

```
if(test){ expr }

if(test){ expr1
                } else{
          expr2 }


ifelse(test, yes, no)
```

Example if statement

We now consider a simple if statement, followed by a second if statement.

```
i=1
if(i==1){ print(”one”) }

i=2
if(i==1){ print(”one”) }
```

What is the output for each if statement?

Example if … else statement

We now consider a simple if statement, followed by a second if statement.

```
i=1 if(i==1){ print(”one”)
        } else {  if(i==2){ print(”two”)
              } else{
                   print(”not one or two”)
                   }
              }
```

What is the output in this case?

Other Loops statements

```
for(var in seq) expr
while(cond) expr
repeat expr
break
next
```

For Loops

```
for(var in seq) expr
```

Lets consider an example where we print the numbers one to ten on the screen

```
for(i in 1:10) {
        print(i)
        }
```

While Loops

```
while(cond) expr    or    while(cond) {expr}
```

Again, lets consider an example where we print the numbers one to ten on the screen:

```
i = 1
while(i <= 10){
        print(i)
        }
```

Repeat Loops

```
repeat expr    or    repeat{expr}
```

Lets consider an example where we print a string five times on the screen:

```
v <- c(”Hello”,” repeat loop”)
cnt <- 0
repeat {  print(v)
          cnt <- cnt+1
          if(cnt >5) {
                       break #see below
                     }
        }
```

In this case, the break statement is activated if `cnt` is greater than 5.

Functions

R functions define a **sequence of instructions** to be re-used
As a rule of thumb, if you are copying and pasting code more than twice then you should be writing a function.

Syntax: `name <- function(arg1,arg2,...){ statement }`

Example:

```
helloworld <- function()print(”Hello World!”)
```

The function is called by typing: `helloworld()`

Functions

Functions are more useful than the previous example. An example of a function which does a simple calculation is shown below:

```
squared <- function(x){ x*x }
```

The last line in the function is the value it returns so for example

```
A=squared(4)
```

will assign the value of 16 to A

Functions tips

It can be convenient to write functions in a separate **.r file**.

Such definition files can be loaded at any point by the source function.

i.e. if I have a text file with functions called `functions.r`, I can run the following code:

```
source(functions.r)
```

References

- Crawley, Michael J. (2013). The R book. Chichester, West Sussex, United Kingdom
- ST4060 - Computer Intensive Statistical Analytics I Lecture notes Eric Wolsztynski eric.w@ucc.ie Statistics, School of Mathematical Sciences, University College Cork, Ireland 2018-2019 Version 1.0
- STAT8010 – Introduction to R course Lecture notes Dr. Justin McGuinness, School of Mathematical, Cork Institute of Technology, Ireland 2019-2020
- Garrett Grolemund and Hadley Wickham 2017,R for Data Science, O'Reilly Media http://r4ds.had.co.nz/ [ISBN: 9781491910399]
- Kabacoff, Robert 2015,R in Action, 2nd Ed., Manning New York [ISBN: 1617291382]
- Norman Matloff 2011,The Art of R Programming, No Starch Press San Francisco [ISBN: 9781593273842]
- Website:R Statistical Programminghttps://www.r-project.org/
- Website:RStudio IDEhttps://www.rstudio.com/